Game-Based Task Offloading and Resource Allocation for Vehicular Edge Computing With Edge-Edge Cooperation

Wenhao Fan, *Member, IEEE*, Mingyu Hua, Yaoyin Zhang, Yi Su, Xuewei Li, Bihua Tang, Fan Wu, *Member, IEEE*, and Yuan'an Liu, *Member, IEEE*

***Abstract*—Vehicular Edge Computing (VEC) enables task offloading from vehicles to the edge servers deployed on Road Side Units (RSUs), thus enhancing the task processing performance of the vehicles. However, in a multi-RSU VEC scenario, the uneven geographical distribution of the vehicles naturally causes the load imbalance among the edge servers and leads to the overload and performance degradation problems of the edge servers in hot areas. To this end, in this paper, we propose a joint task offloading and resource allocation for VEC with edge-edge cooperation, in which the tasks offloaded to a high-load edge server can be further offloaded to the other low-load edge servers. Our objective is to minimize the total task processing delay of all the vehicles while guaranteeing the task processing delay tolerance and the holding time of each vehicle. An M/M/1 queue is used to model the task queuing and task computing processes on each RSU. An exact potential game is adopted to model the competition process for the task offloading among the RSUs. A two-stage iterative algorithm is designed to decompose the optimization problem into two stages and solve them iteratively. We analyze the computational complexity of the algorithm and conduct extensive simulations by varying different crucial parameters. The superiority of our scheme is demonstrated in comparison with 3 other reference schemes.**

***Index Terms*—Edge computing, internet of vehicles, task offloading, resource allocation, game theory.**

I. Introduction

INTERNET of Vehicles (IoV) is an important component of Internet of Things (IoT). Driven by the rapid development of communication and computing technologies, the vehicles in IoV are evolving into smart mobile entities embedded with computing and communicating capabilities, thus supporting various intelligence-enhanced IoV applications, which are usually empowered by the machine learning technologies and are delay-sensitive and computation-intensive. However, owing to the limited resources, the vehicles have to suffer from performance degradation when processing these applications on them locally.

Vehicular Edge Computing (VEC) [1] is a promising approach raised in recent years. The VEC system offloads the tasks of vehicles to the edge servers equipped on or near the Road Side Units (RSUs). The edge servers process these tasks and then send the results back to the vehicles. Compared with the traditional cloud-based scheme [2], [3], [4] that offloads the tasks to the cloud server, the edge-based scheme can provide computing capabilities far more than the vehicles, and a much shorter-distance task transmission than the cloud server.

However, the edge servers still suffer from resource insufficiency problems. The uneven geographical distribution of the vehicles naturally causes load imbalance among the edge servers on different RSUs. When too many tasks are offloaded to an RSU in a hot area, the edge server will become overloaded and critically worsen the QoS provided for the vehicles under the coverage. In a local area network (LAN) where multiple neighboring RSUs are deployed, the high-speed inter-connectivity (e.g. the high-speed fiber connections enabled by the FiWi (Fiber Wireless) technology [5]) among the RSUs can be fully utilized to enable the cooperation among the edge servers. Different from traditional isolated schemes, leveraging this edge-edge cooperation [6], [7], [8], [9], a task offloaded to a high-load edge server can be further offloaded to another neighboring low-load edge server via the wire connection between the two RSUs. In this way, the performance improvement for the VEC system in the LAN can be achieved through load balancing.

Resource management is vital for the VEC system with edge-edge cooperation enabled. Task offloading and resource allocation are two key issues of the optimization problem. Besides, we need to further focus on the inherent problems including the task offloading patterns, the mobility of the vehicles, the task queuing and load balancing on the edge servers, etc. The load balancing and performance improvement by efficiently exploiting the edge-edge cooperation should be also studied. However, the above problems were not fully considered by the existing works in this area. Their load balancing mechanisms are restricted by preconditions and lack of flexibility.

In this paper, to provide a more comprehensive resource management scheme compared with the existing works, we focus on a multi-RSU VEC scenario in an edge-edge-cooperation-enabled LAN and propose a joint task offloading and resource allocation scheme. The work and main contributions of our paper are as follows

1) Focusing on a multi-RSU and multi-task VEC scenario in an edge-edge-cooperation-enabled LAN, we formulate a joint task offloading and resource allocation optimization problem to minimize the total task processing delay of all the vehicles while guaranteeing the task processing delay tolerance of each vehicle and the constraints of the holding time of all the vehicles. The partial task offloading pattern, the task queuing and load balancing on the edge server of each RSU, the task processing delay tolerance of each vehicle, and the holding time of each vehicle staying under the coverage of its RSU are all considered in our scheme. Considering the task queuing problem of the edge servers, we model

Manuscript received 2 April 2022; revised 13 June 2022 and 9 December 2022; accepted 12 January 2023. Date of publication 2 February 2023; date of current version 20 June 2023. This work was supported in part by the National Natural Science Foundations of China under Grants 62271086, 61821001, and 61502050, in part by the Fund of State Key Laboratory of Information Photonics and Optical Communications (Beijing University of Posts and Telecommunications), P. R. China, and in part by Beijing Natural Science Foundation under Grant JQ21036. The review of this article was coordinated by Prof. Kyunghan Lee. *(Corresponding author: Wenhao Fan.)*

The authors are with the School of Electronic Engineering, and Beijing Key Laboratory of Work Safety Intelligent Monitoring, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: whfan@bupt.edu.cn; hmyed85@bupt.edu.cn; zyy1999@bupt.edu.cn; yisu@bupt.edu.cn; lixw@bupt.edu.cn; bhtang@bupt.edu.cn; wufanwww@bupt.edu.cn; yuliu@bupt.edu.cn).

Digital Object Identifier 10.1109/TVT.2023.3241286

0018-9545 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:56:54 UTC from IEEE Xplore. Restrictions apply.

the task queuing and task computing processes on each RSU as an M/M/1 queue based on the queuing theory [10]. To balance the load for the offloaded tasks on all the RSUs, we use the exact potential game theory [11] to model the competition process for the task offloading among the RSUs based on the edge-edge cooperation.

2) To solve our mixed integer nonlinear programming (MINLP) optimization problem efficiently, we designed a two-stage iterative algorithm, which decomposes the problem into a partial task offloading and channel allocation (TOCA) sub-problem and an RSU load balancing (RLB) sub-problem, and then solves them iteratively. In the TOCA sub-problem, we decide the task offloading ratio for each vehicle based on the convexity of the partial task offloading problem, and we design a Hungarian-based heuristic algorithm for efficient wireless channel allocation. In the RLB sub-problem, we first prove the existence of the Nash Equilibrium of the exact potential game by constructing its potential function, and then design a distributed algorithm to solve the problem efficiently. Moreover, we analyze the computational complexity of our algorithm.
3) To evaluate the performance of our scheme, we conduct extensive simulations under seven scenarios by varying different crucial parameters, including different numbers of RSUs, different numbers of high-load RSUs, different computing resources of edge servers, different computing resources of vehicles, different bandwidths of channels, different task data size of vehicles, different task processing delay tolerances of vehicles, and different holding time of vehicles. The superiority of our scheme is demonstrated in comparison with 3 other reference schemes. Moreover, the fast convergence of our scheme is evaluated by analyzing the simulation results.

The rest of this paper is organized as follows: In Section II, we introduce the related works. In Section III, the system model is proposed and explained. The algorithm design is presented in Section IV. The simulations and evaluations are given in Section V. Finally, we conclude our work in Section VI.

II. Related Works

VEC is the edge computing in IoV scenarios. It leverages the resources of the edge servers to enhance the task processing performance of the vehicles under the coverage of the RSUs. Resource management plays a key role in a VEC system. It optimizes the system performance by efficiently scheduling the task offloading and allocating the system resources. The existing research works in this area have done many works, aimed at minimizing the task processing delay [18], [19], [20], the energy consumption [14], [21], [22], [23], the system cost [24], [25], [26], etc. In [14], the authors study the energy-efficient workload offloading problem under the latency constraints and propose a low-complexity distributed solution based on the consensus alternating direction method of multipliers. In [26], a resource management scheme was proposed to introduce both partial computation offloading and reliable task allocation with a reprocessing mechanism. A heuristic algorithm, fault-tolerant particle swarm optimization algorithm was designed for maximizing the reliability (FPSO-MR) with latency constraints.

In a multi-RSU VEC scenario, the geographically uneven distribution of the vehicles may cause load imbalance among the edge servers of the RSUs. The RSUs located in hot areas suffer from overloaded problems, which are brought by the excessive task offloading requests from a large number of vehicles under the coverage, whereas, the RSUs in the cold area have considerable amounts of idle resources that are not fully utilized.

Edge-edge cooperation is a promising solution to the above problem. It enables the task offloading among the edge servers based on the interconnectivity among the RSUs. The load balance can be achieved by carrying out task offloading among the edge servers, thus further improving the system performance. However, most of the existing works studying the load balancing problems only focused on the isolated RSU management, which neglects the edge-edge cooperation and lacks joint resource management for the entire multi-RSU system. The literature [12] considered the binary offloading pattern and concentrated on the offloading decision and computing resources allocation of RSUs to guarantee the real-time QoS for vehicle users. In [13], the partial offloading pattern was taken into consideration. The authors jointly optimized the task processing delay by considering the transmission power and computation capacity of the edge servers. However, both of the two above works neglected the edge-edge cooperation among RSUs.

In [15], edge-edge cooperation was considered in the crossroad scene, it minimized the total task computing delay of vehicles under long-term energy consumption of RSUs by leveraging the Lyapunov optimization method. However, this work only took the edge-edge task offloading as a part of their optimization problem to globally maximize the total system utility. The load balancing, partial task offloading, and the resource allocation are not optimized in this work, thus, the optimization result may provide the vehicles with imbalanced QoS.

The authors in [16] investigated a multi-road site unit selection and resource allocation problem, to minimize the total task offloading delay subject to the bandwidth and computation resources constraints. They proposed two offloading schemes but both of them are restricted by predictions. One is to offload the tasks to the RSU that the vehicles may pass by. It requires the vehicle to drive into the coverage area of the selected RSUs for offloading. The other scheme is to offload the tasks to the RSU that the vehicle will reach through wireless communication between adjacent RSUs which is limited by radio resources and possible multi-hop transmission. This work also did not consider task queuing problem and partial task offloading.

The authors in [17] proposed an SDN-based load balancing task offloading scheme and designed two different game-based algorithms. One of the algorithms is based on the principle "offloading to the nearest server," and the other one predicts task computing delays and then pre-offloads the tasks to the servers where the vehicles will arrive. However, such local-optimization-based load balancing strategies could not fully utilize the edge-edge cooperation among all the RSUs. Moreover, the work did not consider task queuing, partial task offloading, and resource allocation, thus restricting further system performance improvement.

In this paper, we propose a joint task offloading and resource allocation scheme for the multi-RSU VEC system with edge-edge cooperation enabled. The partial task offloading and wireless channel allocation are jointly optimized for the task offloading from the vehicles to the RSUs, where the task queuing processes are considered. For the tasks offloaded to the RSUs, we further conduct load balancing by obtaining the Nash Equilibrium solution to a corresponding exact potential game. In Table I, we summarize the difference between our paper and the other main related works point by point.

III. System Model

The main symbols used in our system model are listed in Table II.

A. Network Model

As shown in Fig. 1, we consider a hierarchical VEC network consisting of N roadside units (RSUs) that cover N different areas. An edge server is deployed on each RSU and all of the servers are inter-communicable by a LAN inter-connecting all the RSUs. We denote the IDs of the edge servers/RSUs as $\mathcal{N} = \{1, 2, \ldots N\}$.

We assume that there exists M_n vehicles in the area of the nth RSU ($n \in \mathcal{N}$), and then we collect these vehicles into a set $\mathcal{M}_n = \{1, 2, \ldots, M_n\}$. Vehicle mn is used to represent the mth vehicle ($m \in \mathcal{M}_n$) covered by RSU n. Generally, in the area covered by each RSU, the traveling vehicles have different positions, driving directions, and speeds. Similar to the works in [27], [28], we model the road where the vehicles are running as a straight line consisting of two lanes with opposite directions. (x_n, y_n) is used to describe the coordinate of RSU

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:56:54 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON BETWEEN OUR SCHEME AND THE MAIN RELATED SCHEMES

Schemes	Edge-Edge Cooperation	Load Balancing	Game Theory-Based Load Balancing	Resource Allocation	Task Queuing	Partial Offloading
[12]	×	×	×	√	×	×
[13]	×	×	×	√	×	√
[14]	×	×	×	√	√	√
[15]	√	×	×	×	√	×
[16]	√	×	×	√	×	×
[17]	√	√	√	×	×	×
Our scheme	√	√	√	√	√	√

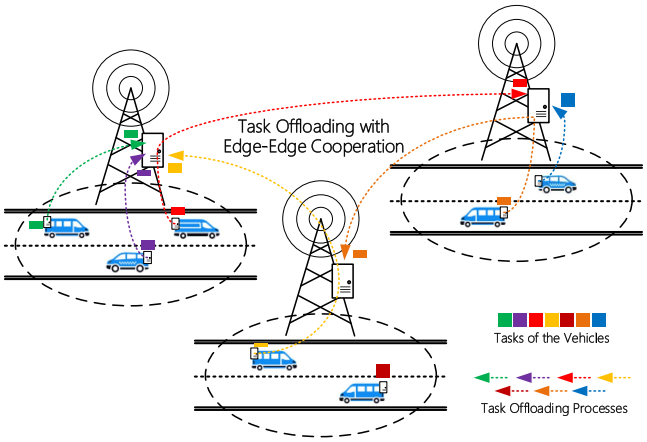


Fig. 1. The topology of the proposed VEC network scenario.

n and (x_{mn}, y_{mn}) is used to describe the initial coordinate of vehicle mn. Thus, the coordinate of vehicle mn after time t can be expressed as $(x_{mn} + v_{mn}t, y_{mn})$, where v_{mn} is the moving speed of the vehicle. Thus, the distance between vehicle mn and RSU n at time t can be formulated as follow:

$$l_{mn}(t) = \sqrt{(x_n - x_{mn} - v_{mn}t)^2 + (y_n - y_{mn})^2 + (z_n - z_{mn})^2} \quad (1)$$

where $z_n - z_{mn}$ is the height difference between the antennas of the vehicle and RSU.

We define the holding time of a vehicle as the duration that the vehicle can maintain the communication with its affiliated RSU before the vehicle moves out of the coverage of the RSU. We denote the holding time of vehicle mn as T_{mn}^{hld}. Note that, "hld" is the abbreviation of "holding". Given D_n as the communication radius of RSU n, T_{mn}^{hld} can be calculated based on the vehicle-RSU distance and the moving speed of the vehicle.

For each vehicle covered by the RSUs, we consider that the applications running on its platform have tasks to be processed. We assume that the task arriving on vehicle mn satisfies a Poisson process with the rate λ_{mn} (in terms of tasks/s). This assumption is valid and has been widely adopted in many works [3], [14], [15]. Further, we profile the attributes of each task of vehicle mn as 1) d_{mn}, the data size (in terms of bits) of the task for task transmission in the task offloading process; 2) $c_{mn} = d_{mn}\eta_{mn}$, the computation amount (in terms of CPU cycles) needed by the task when processed on the vehicle or an edge server. η_{mn} (in terms of CPU cycles/bit) is the computational density of vehicle mn's task, that is, the number of CPU cycles needed to complete the computation of one-bit data; 3) T_{mn}^{tol}, the task processing delay tolerance, which is the maximum delay that the task can accept. It is a constraint indicating the processing delay of the task must not exceed T_{mn}^{tol}. Note that, "tol" is the abbreviation of "tolerance".

As discussed in [29], partial offloading is considered by many research works as a basic task offloading pattern besides binary offloading. Compared with the binary offloading, the partial offloading is more flexible to implement fine-grained task processing. Specifically, the program can be partitioned into two parts with one executed at the vehicle and the other offloaded for edge execution. In our paper, to improve the generalization of the application of our scheme, we consider our partial offloading pattern is based on the widely adopted data-partition model [29], where the task-input bits are bit-wise independent and can be divided into two parts and executed by the vehicle and the edge server, respectively. The two parts run in serial, thus, when the former part is completed on the vehicle, then the latter part starts to be processed on the edge server. Additionally, for the parallel task execution process, similar to the work in [30], our scheme actually optimizes the upper bound of the task processing delay. Thus, our scheme is also applicable for parallel task execution. Through task partitioning, a task can be partially processed on the vehicle locally, and partially offloaded to the edge server of an RSU. Specifically, let α_{mn} be the task offloading ratio to represent the ratio of the offloaded part in the entire task of vehicle mn. We have

$$C1: \alpha_{mn} \in [0, 1], n \in \mathcal{N}, m \in \mathcal{M}_n \quad (2)$$

Thus, $1 - \alpha_{mn}$ indicates the part is arranged to be processed on vehicle mn locally. Note that, when $\alpha_{mn} = 0$, the entire task is processed on the vehicle locally. Contrarily, the task is fully offloaded to an RSU when $\alpha_{mn} = 1$. It can be seen that our scheme includes the binary offloading pattern and represents it by the above two cases.

If a task of a vehicle is decided to be partially offloaded to an RSU, the offloaded part of the task will experience the following steps: (1) task transmission from the vehicle to the RSU that covers it; then, (2) the RSU computes the part, or (3) further offloads it to another RSU for task computing; finally, (4) the result is returned from the RSU to the vehicle. On the side, the other part of the task will be processed on the vehicle locally. Note that, similar to [14], [16], [22], [25], considering the tiny size of the task result, we neglect the task transmission delay of the task result.

B. Computing Model

1) Computing on the Vehicles: We consider the fact that the tasks of vehicle mn will first experience a queuing process before they are computed by the vehicle locally. Thus, based on the queuing theory [31], we model the task computing process on vehicle mn as an M/M/1 queue. Given C_{mn} as the computing resource (in terms of CPU cycles/s) of vehicle mn, we can formulate the task computing delay on the vehicle locally as

$$t_{mn}^{veh} = \frac{(1 - \alpha_{mn})c_{mn}\lambda_{mn}}{C_{mn} - (1 - \alpha_{mn})c_{mn}\lambda_{mn}} \quad (3)$$

where $(1 - \alpha_{mn})c_{mn}\lambda_{mn}$ is the computation amount of the tasks for local processing on vehicle mn.

2) Computing on the RSUs: The tasks offloaded from a vehicle to the RSU that covers it can be further offloaded to another RSU for load balancing. To this end, we introduce a variable β_{mni} to indicate RSU n-RSU i task offloading strategy for vehicle mn. We have

$$C2: \beta_{mni} \in \{0, 1\}, n, i \in \mathcal{N}, m \in \mathcal{M}_n \quad (4)$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:56:54 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SYMBOLS USED IN THE SYSTEM MODEL

Parameter	Description
i, n	RSU index $i, n \in \mathcal{N}$
j	channel index $i, n \in \mathcal{N}$
mn	the mth vehicle ($m \in \mathcal{M}_n$) covered by RSU n
p	transmit powers of the vehicles
h	channel gains
δ^2	background noise power
θ	service rate of the network devices in the LAN
$\mathcal{N}, N$	set / number of RSUs
$\mathcal{M}_n, M_n$	set / number of vehicles
$\mathcal{W}, W$	set / number of radio channels
B	bandwidths of the channels
(x_n, y_n)	coordinate of RSU n
(x_{mn}, y_{mn})	initial coordinate of vehicle mn
h_n, h_{mn}	antenna heights of RSU n and vehicle mn
v_{mn}	moving speed of vehicle mn
t	time
D_n	communication radius of RSU n
T_{mn}^{tol}	task processing delay tolerance of vehicle mn
T_{mn}	processing delay tolerance of vehicle mn's task
T_{mn}^{hld}	holding time of vehicle mn
λ_{mn}	task arrival rate of vehicle mn
d_{mn}	computation data size of vehicle mn's task
c_{mn}	required computation resource of vehicle mn's task
η_{mn}	computational density of vehicle mn's task
α_{mn}	task offloading ratio of vehicle mn
β_{mni}	RSU-RSU task offloading strategy for vehicle mn
γ_{mni}	channel selection for vehicle mn
ψ_{mnj}	SINR on channel j selected by vehicle mn
C_{mn}	computing resource of the vehicle mn
t_{mn}^{veh}	task computing delay of vehicle mn
t_{mn}^{off}	task processing delay via task offloading
t_{mn}	task processing delay of vehicle mn
t_{mni}^{rsu}	task computing delay of the tasks offloaded from vehicle mn to RSU n, and then to RSU i
t_{mn}^{rsu}	task computing delay for the offloaded tasks of vehicle mn
t_{mn}^{v2r}	task transmission delay of vehicle mn transferring its tasks to RSU n
t_{mni}^{r2r}	transmission delay of the tasks of vehicle mn from RSU n to RSU i
t_{mn}^{r2r}	task transmission delay of the tasks of vehicle mn from RSU n to another RSU
r_{mnj}^{v2r}	task transmission rate from vehicle mn to RSU n over the channel
R_{ni}^{r2r}	bandwidth of the connection between RSU n and RSU i

Specifically, $\beta_{mni} = 1$ means the tasks of vehicle mn are further offloaded from RSU n to RSU i, otherwise, $\beta_{mni} = 0$. Note that, $\beta_{mnn} = 1$ represents that the tasks are computed on the original RSU n without RSU-RSU task offloading. We assume that the tasks can not be further partitioned and processed on multiple RSUs owing to the high costs caused by the extra control processes, such as synchronization, coordination, mutual exclusion, etc. Thus, the following constraint must hold

$$C3: \sum_{i \in \mathcal{N}} \beta_{mni} \leq 1, n \in \mathcal{N}, m \in \mathcal{M}_n \tag{5}$$

which means the tasks of vehicle mn can be only computed on the original RSU n or further offloaded to another RSU at most.

Similarly, considering the task queuing on RSU i, we model the task computing process on the RSU as an M/M/1 queue. We denote C_i as the computing resource (in terms of CPU cycles/s) of RSU i. Then, we can formulate the task computing delay of the tasks offloaded from vehicle mn to RSU n, and then to RSU i. It is given by

$$t_{mni}^{rsu} = \frac{\alpha_{mn}\beta_{mni}c_{mn}\lambda_{mn}}{C_i - \sum_{l \in \mathcal{N}} \sum_{k \in \mathcal{M}_l} (\alpha_{kl}\beta_{kli}c_{kl}\lambda_{kl})} \tag{6}$$

where $\alpha_{mn}\beta_{mni}c_{mn}\lambda_{mn}$ is the computation amount of the tasks offloaded from vehicle mn to RSU n, and then to RSU i. $\sum_{l \in \mathcal{N}} \sum_{k \in \mathcal{M}_l} (\alpha_{kl}\beta_{kli}c_{kl}\lambda_{kl})$ is the summed computation amount of all the tasks that are offloaded to RSU i.

Then, we can express the task computing delay for the offloaded tasks of vehicle mn. The formulation is

$$t_{mn}^{rsu} = \sum_{i \in \mathcal{N}} t_{mni}^{rsu} \tag{7}$$

Note that, based on the queuing theory, to guarantee the stability of the task queues, the computation amount of the tasks computed on the vehicles and the RSUs must be less than their computing resources. These constraints can be formulated by

$$C4: C_{mn} - (1 - \alpha_{mn})c_{mn}\lambda_{mn} > 0, \quad m \in \mathcal{M}_n, n \in \mathcal{N} \tag{8}$$

$$C5: C_i - \sum_{l \in \mathcal{N}} \sum_{k \in \mathcal{M}} (\alpha_{kl}\beta_{kli}c_{kl}\lambda_{kl}) > 0, \quad m \in \mathcal{M}_n, n, i \in \mathcal{N} \tag{9}$$

C. Communication Model

1) Task Transmission From the Vehicles to the RSUs: Similar to [32], [33], we assume the radio access system of each RSU is based on OFDMA (Orthogonal Frequency Division Multiple Access). There are total W channels provided by each RSU, and they are collected into a set $\mathcal{W} = \{1, 2, \ldots, W\}$. The bandwidths of the channels are equal, and each of them is denoted by B. Let γ_{mnj} be the channel selection for vehicle mn. We have

$$C6: \gamma_{mnj} \in \{0, 1\}, n \in \mathcal{N}, m \in \mathcal{M}_n, j \in \mathcal{W} \tag{10}$$

$\gamma_{mnj} = 1$ denotes that vehicle mn is selected to use the jth channel ($j \in \mathcal{W}$) for the task transmission from the vehicle to RSU n, otherwise, $\gamma_{mnj} = 0$.

Similar to many research works such as [34], we assume the vehicles are single-antenna communication devices. Thus, a vehicle under an RSU can be arranged with only one channel at most. Correspondingly, a channel of an RSU can be allocated to only one vehicle at most. Thus, the following constraints must hold

$$C7: \sum_{j \in \mathcal{W}} \gamma_{mnj} \leq 1, n \in \mathcal{N}, m \in \mathcal{M}_n, j \in \mathcal{W} \tag{11}$$

$$C8: \sum_{m \in \mathcal{M}_n} \gamma_{mnj} \leq 1, n \in \mathcal{N}, j \in \mathcal{W} \tag{12}$$

Owing to the characteristic of the OFDMA system, there is no interference among the channels of the same RSU, but the co-channel interference among the same channels of different RSUs exists. In this case, the SINR on channel j selected by vehicle mn is expressed as

$$\psi_{mnj} = \frac{p_{mn}h_{mnj}}{I_{nj} + \delta^2} \tag{13}$$

where p_{mn} is the transmit powers of vehicle mn and h_{mnj} is the channel gain. δ^2 is the background noise power. I_{nj} is the interference on channel i brought by all the RSUs except RSU n. I_{nj} is defined by

$$I_{nj} = \sum_{l \in \mathcal{N} \setminus n} \sum_{k \in \mathcal{M}_l} (\gamma_{klj}p_{kl}h_{klj}) \tag{14}$$

The mobility of a vehicle impacts the transmission rate from the vehicle to the RSU because the vehicle is moving on the road during the task transmission process. Based on the work in [35], h_{mnj} can be

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:56:54 UTC from IEEE Xplore. Restrictions apply.

formulated as $h_{mnj} = \zeta_{mj} g_{mn} A(l_{mn}(t))^{-\epsilon}$, where ζ_{mj} is the small-scale fast fading power component, g_{mn} is a log-normal shadow fading, A is the path loss constant and ϵ is the decay exponent.

Given ψ_{mnj}, we can formulate the transmission rate (in terms of bits/s) in the task transmission from vehicle mn to RSU n over channel j as

$$r^{v2r}_{mnj} = B \log_2 (1 + \psi_{mnj}) \tag{15}$$

where "v2r" is the abbreviation of "vehicle to RSU".

As vehicle mn moves, the transmission rate r^{v2r}_{mnj} becomes a time varying value $r^{v2r}_{mnj}(t)$. The data amount needed to be transmitted from vehicle mn to RSU n is $\alpha_{mn} d_{mn} \lambda_{mn}$. Thus, can get the integral equation as follow:

$$\alpha_{mn} d_{mn} \lambda_{mn} = \int_0^{t^{v2r}_{mnj}} r^{v2r}_{mnj}(t) t \mathrm{d}t \tag{16}$$

where t^{v2r}_{mnj} is the transmission delay (in terms of s) of vehicle mn transferring its tasks to RSU n over channel j. $\int_0^{t^{v2r}_{mnj}} r^{v2r}_{mnj}(t) t \mathrm{d}t$ is the sum of the data amount transmitted during t^{v2r}_{mnj}. t^{v2r}_{mnj} can be obtained by solving (16). Owing to the continuity of t^{v2r}_{mnj}, we can use multiple mathematical tools such as MATLAB to calculate its value efficiently.

Then, the task transmission delay of vehicle mn transferring its tasks to RSU n can be formulated as

$$t^{v2r}_{mn} = \sum_{j \in \mathcal{W}} (\gamma_{mnj} t^{v2r}_{mnj}) \tag{17}$$

2) Task Transmission Among the RSUs: When the tasks of the vehicles are further offloaded among the RSUs. The task transmission delay among the RSUs must be taken into account. As aforementioned, all the RSUs are inter-connected in a LAN, where the task transmission also has to experience the task queuing process on the network devices (e.g. switches, routers, etc.). Similarly to [15], [36], we model the task queuing in the LAN as an M/M/1 queue. Let θ be the service rate (in terms of bits/s) of the network devices in the LAN, and let R^{r2r}_{ni} denote the bandwidth of the connection between RSU n and RSU i. "r2r" is the abbreviation of "RSU to RSU". Based on the queuing theory, we have the transmission delay of the tasks of vehicle mn from RSU n to RSU i

$$t^{r2r}_{mni} = \underbrace{\frac{\alpha_{mn} d_{mn} \lambda_{mn}}{\theta - \sum_{l \in \mathcal{N}} \sum_{k \in \mathcal{M}_l} \sum_{z \in \mathcal{N} \setminus l} (\alpha_{kl} \beta_{klz} d_{kl} \lambda_{kl})}}_{\text{part I}} + \underbrace{\frac{\alpha_{mn} d_{mn} \lambda_{mn}}{R^{r2r}_{ni}}}_{\text{part II}} \tag{18}$$

where part I refers to the task queuing delay, and part II represents the transmission delay of the connection between the two RSUs. In part I, $\alpha_{kl} \beta_{klz} d_{kl} \lambda_{kl}$ represents the data size of the tasks of vehicle kl for the task transmission from RSU l to RSU z ($z \neq l$). So $\sum_{l \in \mathcal{N}} \sum_{k \in \mathcal{M}_l} \sum_{z \in \mathcal{N} \setminus l} (\alpha_{kl} \beta_{klz} d_{kl} \lambda_{kl})$ is the data size of all the tasks offloaded among the RSUs.

Thus, we have the task transmission delay of the tasks of vehicle mn from RSU n to another RSU

$$t^{r2r}_{mn} = \sum_{i \in \mathcal{N} \setminus n} (\beta_{mni} t^{r2r}_{mni}) \tag{19}$$

Similarly, based on the queuing theory, the constraint must hold, which represents the total data size in the RSU-RSU task transmission must be less than the service rate of the network devices. Thus, we have

$$C9: \theta - \sum_{l \in \mathcal{N}} \sum_{k \in \mathcal{M}_l} \sum_{z \in \mathcal{N} \setminus l} (\alpha_{kl} \beta_{klz} d_{kl} \lambda_{kl}) > 0 \tag{20}$$

D. Problem Formulation

Because of the serial task execution process, the task processing delay of the tasks of vehicle mn is the sum of two parts: 1) t^{veh}_{mn}, the local task computing delay on vehicle mn; 2) t^{off}_{mn}, the task processing delay via task offloading. t^{off}_{mn} is composed of the task computing delay on the RSU, the Vehicle-RSU task transmission delay, and the RSU-RSU task transmission delay. That is, $t^{off}_{mn} = t^{rsu}_{mn} + t^{v2r}_{mn} + t^{r2r}_{mn}$. Note that, "off" is the abbreviation of "offloading". Thus, the task processing delay of vehicle mn is

$$\begin{aligned} t_{mn} &= t^{veh}_{mn} + t^{off}_{mn} \\ &= t^{veh}_{mn} + t^{rsu}_{mn} + t^{v2r}_{mn} + t^{r2r}_{mn} \end{aligned} \tag{21}$$

As aforementioned, t_{mn} must not exceed the task processing delay tolerance T^{tol}_{mn} of vehicle mn. That is

$$C10: t_{mn} \leq \lambda_{mn} T^{tol}_{mn}, \quad m \in \mathcal{M}_n, n \in \mathcal{N} \tag{22}$$

For the offloaded tasks of vehicle mn, the task processing delay t^{off}_{mn} must not exceed the holding time T^{hld}_{mn}, namely, the task result needs to be returned from the RSU to the vehicle before it moves out of the coverage of the RSU. This constraint can be written as

$$C11: t^{off}_{mn} \leq \lambda_{mn} T^{hld}_{mn}, \quad m \in \mathcal{M}_n, n \in \mathcal{N} \tag{23}$$

Finally, the total task processing delay of the vehicles in all the RSUs can be formulated as

$$t = \sum_{n \in \mathcal{N}} \sum_{m \in \mathcal{M}_n} t_{mn} \tag{24}$$

Let $\boldsymbol{\alpha} = \{\alpha_{mn}\}_{n \in \mathcal{N}, m \in \mathcal{M}_n}$, $\boldsymbol{\beta} = \{\beta_{mni}\}_{n,i \in \mathcal{N}, m \in \mathcal{M}_n}$, and $\boldsymbol{\gamma} = \{\gamma_{mnj}\}_{n \in \mathcal{N}, m \in \mathcal{M}_n, j \in \mathcal{W}}$ be three sets containing all the indicator variables. The objective of our scheme is to minimize the total task processing delay of all the vehicles while guaranteeing the task processing delay tolerance of each vehicle and the constraints of the holding time of all the vehicles. Thus, we construct our optimization problem as follow:

$$\begin{aligned} \mathcal{P}1: \ &\underset{\boldsymbol{\alpha}, \boldsymbol{\beta}, \boldsymbol{\gamma}}{\text{minimize}} \ t \\ &\text{s.t.} \quad C1 - C11 \end{aligned} \tag{25}$$

where $C1$-$C11$ are explained previously.

IV. Two-Stage Iterative Algorithm Design

$\mathcal{P}1$ is a complex MINLP problem since (1) the variables in $\boldsymbol{\alpha}$ are continuous, whereas, the variables in $\boldsymbol{\beta}$ and $\boldsymbol{\gamma}$ are discrete; (2) the objective function and constraints $C5$ and $C19 - C11$ are all nonlinear. Moreover, the variables of $\mathcal{P}1$ are high-dimensional and deeply coupled with each other. The optimal solution to $\mathcal{P}1$ can be solved via the exhaustive search method, which enumerates all optional combinations of the variables. However, owing to the high computational complexity, this method is extremely time-consuming, high-cost, and impractical for real VEC systems. To this end, we propose a two-stage iterative algorithm to handle the optimization efficiently.

Intuitively, the entire task offloading process for the tasks of a vehicle consists of two parts: the vehicle-RSU sub-process and the RSU-RSU sub-process. To naturally coincide with the two sub-processes, the main idea of our algorithm is to decompose the solving process into two stages. In the first stage, we solve the partial task offloading and channel allocation (TOCA) sub-problem to decide the task offloading ratio for the tasks of each vehicle and decide the channels allocated to each vehicle for the task offloading. Then, in the second stage, we solve the RSU load balancing (RLB) sub-problem to realize load balance among the offloaded tasks on all the RSUs by deciding the RSU-RSU task offloading strategies for these tasks.

Essentially, in the first stage, we disable the edge-edge cooperation and then minimize the total task processing delay of the tasks of all the vehicles covered by each RSU by solving the TOCA sub-problem. In

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:56:54 UTC from IEEE Xplore. Restrictions apply.

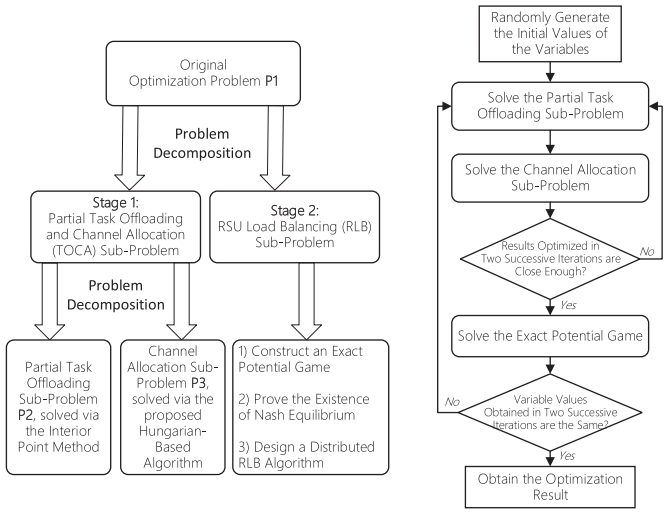


Fig. 2. Problem Decomposition and Flowchart of the Two-Stage Iterative Algorithm.

the second stage, to further reduce the total task processing delay of the offloaded tasks on all the RSUs, we enable the edge-edge cooperation to conduct the load balancing among all the RSUs by solving the RLB sub-problem.

The TOCA and RLB sub-problems are solved iteratively until the algorithm converges, and then the sub-optimal solution to $\mathcal{P}1$ will be obtained via the two stages of our algorithm.

To solve the TOCA sub-problem, we further decompose the sub-problem into two sub-problems: (1) partial task offloading sub-problem. We prove its convexity and then solve it via the interior point method; (2) channel allocation sub-problem. We design a Hungarian-based algorithm to solve it by searching for the proper matches between the vehicles and the channels.

For the RLB sub-problem, we propose a corresponding exact potential game for the RSU-RSU task offloading of the tasks of all the vehicles. Every player in the game aims at minimizing its task processing delay. Then, we construct a potential function, based on which we further prove the existence of the Nash Equilibrium (NE) of the game. Thus, the RLB sub-problem is solved by finding the load-balanced RSU-RSU task offloading strategy that reaches the NE point of the game.

The problem decomposition and flowchart of the two-stage iterative algorithm are shown in Fig. 2.

Now, we explain the details of our algorithm as follows:

A. Stage 1: Partial Task Offloading and Channel Allocation (TOCA) Sub-Problem

1) Task Partial Offloading Sub-Problem: When the values of the variable in $\boldsymbol{\beta}$ and $\boldsymbol{\gamma}$ are given (denoted by $\overline{\boldsymbol{\beta}} = \{\overline{\beta}_{mni}\}_{n,i\in\mathcal{N},m\in\mathcal{M}_n}$ and $\overline{\boldsymbol{\gamma}} = \{\overline{\gamma}_{mnj}\}_{n\in\mathcal{N},m\in\mathcal{M}_n,j\in\mathcal{W}}$, respectively), we can rewritten $\mathcal{P}1$ as the partial task offloading sub-problem $\mathcal{P}2$ with regard to $\boldsymbol{\alpha}$. We have

$$\begin{aligned}\mathcal{P}2:\ \min_{\boldsymbol{\alpha}}\Bigg(&\sum_{n\in\mathcal{N}}\sum_{m\in\mathcal{M}_n}\frac{(1-\alpha_{mn})\,c_{mn}\lambda_{nm}}{C_{mn}-(1-\alpha_{mn})\,c_{mn}\lambda_{nm}}\\&+\sum_{i\in\mathcal{N}}\frac{\sum_{n\in\mathcal{N}}\sum_{m\in\mathcal{M}_n}(\alpha_{mn}\overline{\beta}_{mni}c_{mn}\lambda_{mn})}{C_i-\sum_{l\in\mathcal{N}}\sum_{k\in\mathcal{M}_l}(\alpha_{kl}\overline{\beta}_{kli}c_{kl}\lambda_{kl})}\\&+\sum_{n\in\mathcal{N}}\sum_{m\in\mathcal{M}_n}\left(\alpha_{mn}d_{mn}\lambda_{mn}\sum_{j\in\mathcal{W}}\frac{\overline{\gamma}_{mnj}}{r_{mnj}}\right)\\&+\frac{\sum_{n\in\mathcal{N}}\sum_{m\in\mathcal{M}_n}\sum_{i\in\mathcal{N}\setminus n}(\alpha_{mn}\overline{\beta}_{mni}d_{mn}\lambda_{mn})}{\theta-\sum_{l\in\mathcal{N}}\sum_{k\in\mathcal{M}_l}\sum_{z\in\mathcal{N}\setminus l}(\alpha_{kl}\overline{\beta}_{klz}d_{kl}\lambda_{kl})}\\&+\sum_{n\in\mathcal{N}}\sum_{m\in\mathcal{M}_n}\sum_{i\in\mathcal{N}\setminus n}\frac{\alpha_{mn}\overline{\beta}_{mni}d_{mn}\lambda_{mn}}{R_{ni}^{r2r}}\Bigg)\end{aligned}$$

$$\text{s.t. } C1, C4, C5, C9-C11 \tag{26}$$

where the objective function is derived from the objective function of the problem $P1$ by removing the parts that are irrelevant with the variables in $\boldsymbol{\alpha}$.

According to [37], we can prove that $\mathcal{P}2$ is a convex optimization problem since 1) the objective function is a positive sum of multiple convex functions with respect to $\boldsymbol{\alpha}$ and 2) the constraints $C1$, $C4$, $C5$, and $C9-C11$ are linear or convex functions with respect to $\boldsymbol{\alpha}$. Based on the convexity and nonlinearity of $\mathcal{P}2$, this sub-problem can be solved efficiently via a proper convex optimization algorithm. Here, we use the interior point method [37], which has a $O(|\boldsymbol{\alpha}|^3)$ computational complexity. Note that, $|\boldsymbol{\alpha}| = \sum_{n\in\mathcal{N}} M_n$ is the number of the variables in $\boldsymbol{\alpha}$.

2) Channel Allocation Sub-Problem: In $\mathcal{P}1$, the variables in $\boldsymbol{\beta}$ and $\boldsymbol{\gamma}$ are naturally decoupled. When the values of the variables in $\boldsymbol{\alpha}$ are given (denoted by $\overline{\boldsymbol{\alpha}} = \{\overline{\alpha}_{mn}\}_{n\in\mathcal{N},m\in\mathcal{M}_n}$), we can written $\mathcal{P}1$ as the channel allocation sub-problem $\mathcal{P}3$ with regard to $\boldsymbol{\gamma}$. We have

$$\begin{aligned}\mathcal{P}3:\ \min_{\boldsymbol{\gamma}} H(\boldsymbol{\gamma}) &= \sum_{n\in\mathcal{N}}\sum_{m\in\mathcal{M}_n}\sum_{j\in\mathcal{W}}\left(\gamma_{mnj}t_{mnj}^{v2r}(\overline{\alpha}_{mn})\right)\\ \text{s.t. } &C6,\ C7,\ C8\end{aligned} \tag{27}$$

where $t_{mnj}^{v2r}(\overline{\alpha}_{mn})$ is obtained by substituting $\alpha_{mn} = \overline{\alpha}_{mn}$ to t_{mnj}^{v2r}. $\mathcal{P}3$ belongs to a nonlinear binary programming problem. Solving it is equivalent to finding proper matches between vehicles and channels. In the OFDMA system, there is no interference among the channel of the same RSU. However, the co-channel interference among the same channel of different RSUs exists. It makes the channel allocation of all the RSUs couple with each other. The change in the channel allocation for any RSU may impact the channel allocation strategies for the other RSUs. To this end, we design a Hungarian-based algorithm to solve $\mathcal{P}3$ efficiently through cooperating among all the RSUs.

Coincided with the characteristic of $\mathcal{P}3$, the main idea of our algorithm is based on an iterative process, which solves the channel allocation sub-problem for each RSU separately by fixing the co-channel interference, selects the best channel allocation strategy from those for all the RSUs, and then globally updates the co-channel interference if the channel allocation strategy for any RSU changes. This process converges until no new channel allocation strategy for any RSU can decrease the total task transmission delay.

Initially, we ignore the co-channel interference among all the RSUs. Then, the channel allocation sub-problem can be solved for each RSU separately. We can rewrite $\mathcal{P}3$ as the following channel allocation sub-problem $\mathcal{P}4$ for RSU n

$$\begin{aligned}\mathcal{P}4:&\min_{\boldsymbol{\gamma}_n}\sum_{m\in\mathcal{M}_n}\sum_{j\in\mathcal{W}}\left(\gamma_{mnj}t_{mnj}^{v2r'}(\overline{\alpha}_{mn})\right)\\ &\text{s.t. } C6': \gamma_{mnj}\in\{0,1\}, m\in\mathcal{M}_n, j\in\mathcal{W}\\ &C7': \sum_{j\in\mathcal{W}}\gamma_{mnj}\le 1, m\in\mathcal{M}_n, j\in\mathcal{W}\\ &C8': \sum_{m\in\mathcal{M}_m}\gamma_{mnj}\le 1, j\in\mathcal{W}\end{aligned} \tag{28}$$

where $\boldsymbol{\gamma}_n = \{\gamma_{mnj}\}_{m\in\mathcal{M}_n,j\in\mathcal{W}}$ is the channel allocation indicators for RSU n. $t_{mnj}^{v2r'}(\overline{\alpha}_{mn})$ is $t_{mnj}^{v2r}(\overline{\alpha}_{mn})$ with I_{nj} removed. $\mathcal{P}4$ is a task assignment problem and can be solved efficiently via the Hungarian

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:56:54 UTC from IEEE Xplore. Restrictions apply.

method. For each vehicle under RSU n, we compute the task transmission delays of the vehicle over all the channels of the RSU (e.g. t^{v2r}_{mnj} of vehicle mn over channel j. See (16)). These delays are used as the weights of the edges in the bigraph constructed by the vehicles and the channels. In the bigraph, the Hungarian method aims at finding the maximum matching that minimizes the total transmission delay of all the vehicles in the RSU.

Based on the initialized channel allocation strategies of all the RSUs, we further compute the co-channel interference on each channel of all the RSUs. (e.g. I_{nj}, the co-channel interference for channel j of RSU n. See (14)). Then, using the Hungarian method, we solve the channel allocation sub-problem for each RSU with the co-channel interference substituted. Specifically, the sub-problem for RSU n is formulated as

$$\mathcal{P}5: \min_{\boldsymbol{\gamma}_n} \sum_{m\in\mathcal{M}_n}\sum_{j\in\mathcal{W}}\left(\gamma_{mnj}t^{v2r}_{mnj}(\overline{\alpha}_{mn})\right)$$
$$\text{s.t. } C6,' C7,' C8' \tag{29}$$

Then, we compare the total task transmission delay of the tasks offloaded from all the RSUs (denoted by $H(\boldsymbol{\gamma})$. See (27)) with or without the new channel allocation strategy for each RSU substituted. If the strategy that makes the largest performance improvement is better than the performance without substitution over a certain extent (denoted by ϵ), the strategy will be updated and the algorithm goes to the next iteration starting at the co-channel interference computing process. Otherwise, our algorithm stops and outputs the previously updated channel allocation strategies as the final result.

The details of our Hungarian-based algorithm is described in Algorithm 1, where $iter$ is the iterator of the loop. $\boldsymbol{\gamma}^{(iter)}$ refers to the channel allocation strategies of all the RSUs obtained in the $iter$th iteration, and $\boldsymbol{\gamma}^{(iter)}\backslash\boldsymbol{\gamma}_n^{(iter)} \cup \{\boldsymbol{\gamma}_n^{(iter+1)}\}$ is the strategies with the strategy of RSU n in the $iter+1$th iteration replaced.

The complexity of the Hungarian algorithm used to solve $\mathcal{P}4$ and $\mathcal{P}5$ is $O(W^3)$. Let τ denote the number of the times that the Hungarian algorithm is executed in our Hungarian-based channel allocation algorithm. τ satisfies the inequality: $(1+\epsilon)^\tau < \frac{RST}{INT}$, where RST represents the value of $H(\boldsymbol{\gamma}^*)$, and INT is the value of $H(\boldsymbol{\gamma}^{(0)})$. Thus, $\tau = \log_{(1+\epsilon)}\frac{RST}{INT}$. Therefore, the complexity of our Hungarian-based channel allocation algorithm is $O(W^3\log_{(1+\epsilon)}\frac{OPT}{NOT})$.

Algorithm 1: Hungarian-Based Channel Allocation Algorithm.

Initialization:
  for each RSU $n \in \mathcal{N}$
    Solve $\mathcal{P}4$ via the Hungarian method, and then obtain $\boldsymbol{\gamma}_n^{(0)}$;
  end for
  $\boldsymbol{\gamma}^{(0)} = \cup_{n\in\mathcal{N}}\{\boldsymbol{\gamma}_n^{(0)}\}$;
Iterative Process:
  $iter = 0$;
  loop
    Obtain $H^{(iter)} = H(\boldsymbol{\gamma}^{(iter)})$;
    for each channel $j \in \mathcal{W}$ of RSU $n \in \mathcal{N}$ **then**
      Compute the co-channel interference I_{nj} with $\boldsymbol{\gamma}^{(iter)}$ substituted;
    end for
    for each RSU $n \in \mathcal{N}$
      Solve $\mathcal{P}5$ via the Hungarian method, and then obtain $\boldsymbol{\gamma}_n^{(iter+1)}$;
      Obtain $H_n^{(iter+1)} = H(\boldsymbol{\gamma}^{(iter)}\backslash\boldsymbol{\gamma}_n^{(iter)} \cup \{\boldsymbol{\gamma}_n^{(iter+1)}\})$
    end for
    $p = \arg\ \max_{p\in\mathcal{N}} H_p^{(iter+1)}$; // find the RSU whose strategy makes the largest performance improvement
    if $(1+\epsilon)H_p^{(iter+1)} \le H^{(iter)}$ **then**
      $\boldsymbol{\gamma}^{(iter+1)} = \boldsymbol{\gamma}^{(iter)}\backslash\boldsymbol{\gamma}_n^{(iter)} \cup \{\boldsymbol{\gamma}_n^{(iter+1)}\}$
      $iter = iter + 1$;
    else
      quit;
    end if
  end loop
Output $\boldsymbol{\gamma}^* = \boldsymbol{\gamma}^{(iter)}$;

B. Stage 2: RSU Load Balancing (RLB) Sub-Problem

When the partial task offloading strategies (denoted by $\overline{\boldsymbol{\alpha}} = \{\overline{\alpha}_{mn}\}_{n\in\mathcal{N},m\in\mathcal{M}_n}$) of all the vehicles are decided in stage 1, the tasks of the vehicles are partially offloaded to the RSU that covers them. These tasks can be computed on the RSU or further offloaded to the other RSUs via the edge-edge cooperation. The RLB sub-problem corresponds to the load balancing process among the RSUs. It can be modeled as a competition process, where each RSU tries to assist the tasks offloaded from the vehicles covered by it to minimize their task processing delays until the system performance can not be further improved. To this end, we formulate this process as an exact potential game, prove the Nash Equilibrium (NE) of the game by constructing a corresponding potential function, and then design a distributed RLB (DRLB) algorithm to solve the game efficiently through the iterative interactions among all the RSUs.

1) Game Formulation: We formulate the RLB sub-problem as a strategic game $\mathcal{G} = \langle\mathcal{V}, \boldsymbol{\beta}, \{u_{mn}\}_{n\in\mathcal{N},m\in\mathcal{M}_n}\rangle$. The set $\mathcal{U}$ includes all the players of the games, namely, each player represents the offloaded tasks of each vehicle under each RSU. We use the RSU-RSU task offloading indicator set $\boldsymbol{\beta}$ (defined in Section III.D) to denote the feasible strategy set of the offloaded tasks of all the vehicles. Let $\boldsymbol{\beta}_{mn}$ be the strategy set for the offloaded tasks of vehicle mn, and we have $\boldsymbol{\beta}_{mn} = \{\beta_{mni}\}_{i\in\mathcal{N}}$. u_{mn} is a utility function of the offloaded tasks of vehicle mn. It measures the tasks' processing delay on the RSUs and is formulated as

$$u_{mn}(\beta_{mn}, \boldsymbol{\beta}_{-mn}) = t^{rsu}_{mn}(\overline{\alpha}_{mn}) + t^{r2r}_{mn}(\overline{\alpha}_{mn})$$
$$= \underbrace{\sum_{i\in\mathcal{N}} \frac{\overline{\alpha}_{mn}\lambda_{mn}\beta_{mni}c_{mn}}{C_i - \sum_{l\in\mathcal{N}}\sum_{k\in\mathcal{M}_l}(\alpha_{kl}\beta_{kli}c_{kl}\lambda_{kl})}}_{\text{part I}}$$
$$+ \underbrace{\sum_{i\in\mathcal{N}\backslash n}\left(\frac{\overline{\alpha}_{mn}\lambda_{mn}\beta_{mni}d_{mn}}{\theta - \sum_{l\in\mathcal{N}}\sum_{k\in\mathcal{M}_l}\sum_{z\in\mathcal{N}\backslash l}(\alpha_{kl}\beta_{klz}d_{kl}\lambda_{kl})}\right.}_{\text{part II}}$$
$$\underbrace{\left. + \frac{\overline{\alpha}_{mn}\lambda_{mn}\beta_{mni}d_{mn}}{R^{r2r}_{ni}}\right)}_{\text{part II}} \tag{30}$$

where part I (i.e., the expansion of $t^{rsu}_{mn}(\overline{\alpha}_{mn})$) is the task processing delay on the RSUs, and part II (i.e., the expansion of $t^{r2r}_{mn}(\overline{\alpha}_{mn})$) is the RSU-RSU task transmission delay if the tasks are further offloaded to another RSU.

In the game process, each player chooses a strategy from its strategy set to minimize its utility function. For example, the player mn representing the offloaded tasks of vehicle mn chooses a strategy β_{mn} from the set $\boldsymbol{\beta}_{mn}$ to minimize its utility function $u_{mn}(\beta_{mn}, \boldsymbol{\beta}_{-mn})$. The set $\boldsymbol{\beta}_{-mn}$ contains the strategies chosen by the other players except for the player mn. When all the players have chosen their strategies, all the strategies will be updated and sent to all the players. Then, each player will choose its strategy again based on the updated strategies of the other players.

The game $\mathcal{G}$ is an exact potential game. The proof is shown in Appendix A.

Further, we can prove that the NE of the game $\mathcal{G}$ exists. We give the proof in Appendix B.

2) Distributed RLB (DRLB) Algorithm: We propose a DRLB algorithm to solve the RLB sub-problem through finding the NE of the proposed exact potential game $\mathcal{G}$ with the cooperation of all the RSUs.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:56:54 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: DRLB Algorithm.

Initialization:
Randomly generate the initial strategy $\boldsymbol{\beta}^{(prv)}$ for all the players;
Exact Potential Game:
loop
$\boldsymbol{\beta}^{(nxt)} = \boldsymbol{\beta}^{(prv)}$;
for each RSU $n \in \mathcal{N}$ **do**
for each player $mn, m \in \mathcal{M}_n$ **then**
Solve $\mathcal{P}6$ with $\boldsymbol{\beta}^{(iter)}$ substituted, and then obtain β'_{mn};
end for
end for
Collect all the strategy improvements into the set Ω;
loop
Solve $\mathcal{P}7$ with Ω substituted, and then obtain β;
if $u(\boldsymbol{\beta}^{(nxt)} \backslash \beta^{(nxt)} \cup \beta) < u(\boldsymbol{\beta}^{(prv)})$ **then**
Update β into $\boldsymbol{\beta}^{(nxt)}$;
else
quit;
end if
end loop
if $\boldsymbol{\beta}^{(nxt)} = \boldsymbol{\beta}^{(prv)}$ **then**
quit;
end if
end loop
Output: $\boldsymbol{\beta}^{(nxt)}$.

We design a collaborative RSU-RSU task offloading mechanism as follows:

- Through playing the proposed potential game, the RSUs collaboratively determine where the offloaded tasks of each vehicle are processed. Each RSU takes the responsibility to choose the optimal strategy improvement for each player, that is, the offloaded tasks of each vehicle under its coverage.
- At the beginning of each round of the game, the RSUs broadcast the information about their workloads and available computing resources. By analyzing the information, the RSUs will choose the optimal offloading strategy for each player according to (30). Then these strategies will be broadcast to the other RSUs.
- The RSUs collect the strategy improvement broadcasts and then take the strategy improvement that can minimize the value of the sum of all players' utility functions. Moreover, the RSUs determine whether the game reaches the NE based on the result obtained in each round of the game.

We introduce the details of the DRLB algorithm in Algorithm 2, where we use nxt and prv as the iterators of the loop.

At the beginning of the game, each RSU calculates the value of the utility function of each player with each strategy in the strategy set substituted. Then, the RSU chooses the strategy, that minimizes the utility function of each player, as the strategy improvement. For the player mn, its strategy improvement β'_{mn} is obtained by solving the following optimization problem:

$$\mathcal{P}6: \beta'_{mn} = \underset{\beta_{mn} \in \boldsymbol{\beta}_{mn}}{\arg\min}\ u_{mn}(\beta_{mn}, \boldsymbol{\beta}_{-mn}) \tag{31}$$

The RSUs collect the strategy improvements for all the players in the game, and then substitute these strategy improvements into $u = \sum_{n \in \mathcal{N}} \sum_{m \in \mathcal{M}} u_{mn}$, which is the sum of the utility functions of all the players. Then, the RSUs evaluate whether a strategy improvement can reduce u. If so, they update the strategy of the corresponding player to the strategy improvement, otherwise, the strategy keeps unchanged. Note that, when multiple strategy improvements can meet the criterion, a greedy decision process will be taken to choose the ones that can reduce u by the largest extent, that is, it is equivalent to solving the following optimization problem:

$$\mathcal{P}7: \beta = \underset{\beta \in \Omega}{\arg\min}\ u \tag{32}$$

where Ω is the set of the collected strategy improvements. Note that, the utility function of each player will be impacted when a strategy improvement is updated. Hence, u needs to be recalculated correspondingly before evaluating the other strategy improvements.

The rounds of the game continue until there is no strategy improvement can be found to reduce u. That is, if the updated strategy improvements in the current round are the same as the ones in the previous round, the game will reach NE, and the obtained $\boldsymbol{\beta}$ is the final solution of the RLB sub-problem. Otherwise, the aforementioned process will repeat until the game ends with the NE.

In the DRLB algorithm, each RSU is in charge of calculating and comparing the utility functions of its players. The complexity of the process on RSU n is $O(M_n)$. The RSU is also in charge of evaluating the strategy improvement broadcasts. The complexity of the process can be estimated as $O(N)$. Assuming the number of the rounds needed by the game is π, we can estimate the complexity of the game on each RSU as $O(\pi(M_n + N))$ at most.

C. Procedure of the Two-Stage Iterative Algorithm

Our two-stage iterative algorithm has been described at the beginning of Section IV, and its flowchart is shown in Fig. 2. Here, we give the details of the algorithm in Algorithm 3. The entire algorithm is composed of an inner loop and an outer loop. We use a and b as the iterators of the inner loop and the outer loop, respectively. In the inner loop, we solve the TOCA sub-problem. The partial task offloading sub-problem and the channel allocation sub-problem are solved iteratively. ι is a very small value that controls the termination condition of the inner iteration process. In the outer loop, we solve the RLB sub-problem. The termination condition is to check whether the obtained solutions in the last two successive iterations are the same. Finally, the solution to the entire problem is obtained when the termination condition of the out loop is satisfied.

D. Algorithm Deployment and Execution

Our two-stage iterative algorithm is deployed on the edge server of each RSU, thus it runs in a distributed manner. Based on the collected network state information and task information, in Stage 1, each edge server is in charge of solving the TOCA sub-problem for the tasks of all the vehicles under its coverage; in Stage 2, by running the DRLB algorithm, each edge server acts as a player and plays the exact potential game $\mathcal{G}$ iteratively with the other edge servers. Finally, the task offloading decision for each vehicle (i.e. the optimized variable in $\boldsymbol{\alpha}$) is treated as control information and then is delivered to the vehicle by the RSU that covers it. The RSU-RSU task offloading decisions and the channel allocation decisions for each RSU (i.e. the optimized variables in $\boldsymbol{\beta}$ and $\boldsymbol{\gamma}$, respectively) are directly obtained by the RSU.

Note that, similar to many works such as [38], [39], the network state information, task information, and control information are usually very tiny, their transmission delays bring very low costs and can be ignored in the optimization. In addition, the needed information can also be transmitted along with the task offloading process as the protocol fields embedded into the task data, or be directly obtained from the system state information service running in the system management system (e.g. the digital twin system). Thus, in this way, the costs can be eliminated.

V. Simulations and Evaluations

To evaluate the performance of our scheme in detail, we conduct extensive simulations in seven different scenarios, where we change the number of RSUs, the number of high-load RSUs, the computing resources of edge servers, the computing resources of vehicles, the bandwidths of channels, the task data size of

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:56:54 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: Two-Stage Iterative Algorithm.

Initialization:

Randomly generate $\boldsymbol{\alpha}^{(0)}$, $\boldsymbol{\beta}^{(0)}$ and $\boldsymbol{\gamma}^{(0)}$ as the initial values of the variables in $\boldsymbol{\alpha}$, $\boldsymbol{\beta}$ and $\boldsymbol{\gamma}$ while guaranteeing the constraints $C4-C11$;

Iterative Process:

$a=0$;

loop

Stage 1: Solve the TOCA sub-problem:

$b=0$;

$\boldsymbol{\alpha}^{(a)(b)}=\boldsymbol{\alpha}^{(a)}, \boldsymbol{\gamma}^{(a)(b)}=\boldsymbol{\gamma}^{(a)}$;

loop

$\boldsymbol{\beta}^{(a)(b)}=\boldsymbol{\beta}^{(a)}$;

Solve the partial task offloading sub-problem via the interior point method with $\boldsymbol{\beta}^{(a)(b)}$ and $\boldsymbol{\gamma}^{(a)(b)}$ substituted, and then obtain $\boldsymbol{\alpha}^{(a)(b+1)}$;

Solve the channel allocation sub-problem via the Algorithm 1 with $\boldsymbol{\alpha}^{(a)(b+1)}$ and $\boldsymbol{\beta}^{(a)(b)}$ substituted, and then obtain $\boldsymbol{\gamma}^{(a)(b+1)}$;

if $|t(\boldsymbol{\alpha}^{(a)(b+1)}, \boldsymbol{\beta}^{(a)(b+1)}, \boldsymbol{\gamma}^{(a)(b+1)}) - t(\boldsymbol{\alpha}^{(a)(b)}, \boldsymbol{\beta}^{(a)(b)}, \boldsymbol{\gamma}^{(a)(b)})| \leq \iota$ **then**

quit;

end if

$b=b+1$;

end loop

$\boldsymbol{\alpha}^{(a+1)}=\boldsymbol{\alpha}^{(a)(b+1)}, \boldsymbol{\gamma}^{(a+1)}=\boldsymbol{\gamma}^{(a)(b+1)}$;

Stage 2: Solve the RLB sub-problem via the Algorithm 2 with $\boldsymbol{\alpha}^{(a+1)}$ and $\boldsymbol{\gamma}^{(a+1)}$, and then obtain $\boldsymbol{\beta}^{(a+1)}$;

if $\boldsymbol{\beta}^{(a+1)}=\boldsymbol{\beta}^{(a)}$ **then**

quit;

end

end loop

Output: $\boldsymbol{\alpha}^{*}=\boldsymbol{\alpha}^{(a+1)}$, $\boldsymbol{\beta}^{*}=\boldsymbol{\beta}^{(a+1)}$ and $\boldsymbol{\gamma}^{*}=\boldsymbol{\gamma}^{(a+1)}$.

vehicles, the task processing delay tolerances of vehicles, and the holding time of vehicles. Moreover, we also evaluate the fast convergence of our scheme by analyzing the simulation results.

The simulation parameters are selected based on the research works [5], [12], [23]. The simulation result in each scenario is averaged from 200 simulations. The radius of the coverage of each RSU is set between 200–300 m. The locations, moving speeds, moving directions of the vehicles covered by each RSU are configured based on the statistics that we monitored from the traffic flow (from 13:00-15:00 and 17:30–19:30) in 10 locations around Beijing University of Posts and Telecommunications, Beijing, China. Based on the above information, we can calculate the holding time of each vehicle covered by each RSU in our simulations. Same as [35], in the simulations, we consider the path loss model is $128.1+37.6\lg l_{mn}(t)$[km], the shadowing distribution is Log-normal, the shadowing standard deviation is 8 dB, and the fast fading is Rayleigh fading. All the simulation parameters are valued from Table III unless stated elsewhere clearly. Each M_n, d_{mn}, η_{mn}, λ_{mn}, T_{mn}^{tol}, C_m, c_{mn}, R_{ni}^{r2r}, $n\in\mathcal{N}, m\in\mathcal{M}_n, j\in\mathcal{W}$ are randomly selected from their corresponding value ranges in the table to embody the diversity of the tasks and resources in the scenario.

We evaluate the performance of our scheme in comparison with 4 other schemes:

1) *Binary task offloading and channel allocation scheme (BOCA):* the tasks of the vehicles are entirely processed by themselves locally, or entirely offloaded to the RSUs that cover them. Meanwhile, the channel allocation of each RSU is optimized. The edge-edge cooperation is disabled in this scheme, so RSU-RSU task offloading and load balancing can not be used for system performance improvement. This scheme can embody the idea of the scheme in [12].

TABLE III
PARAMETERS USED IN SIMULATIONS AND THEIR DEFAULT VALUES

Parameter	Value
Number of RSUs, N	5
Number of vehicles covered by a RSU, M_n	$10\sim 50$
Tasks data size of a vehicle, d_{mn}	$10\sim 20$ MB
Computational density of the task of a vehicle, η_{mn}	$[0.008, 0.01]$ CPU cycles/bit
Task arrival rate of a vehicle, λ_{mn}	$0.5\sim 1.5$ tasks/s
Delay tolerance of a vehicle, T_{mn}^{tol}	$3\sim 8$ s
Computing resource of an RSU, C_m	$20\sim 40$ G CPU cycles/s
Computing resource of a vehicle, c_{mn}	$1.6\sim 3.2$ G CPU cycles/s
Transmission power of a vehicle, p_{mn}	23 dBm
Noise power, δ^2	-97 dBm
Channel bandwidth, B	20 MHz
Service rate of the LAN, θ	2 GB/s
Bandwidth of the connection from RSU i to n, R_{ni}^{r2r}	$6\sim 10$ Gbps
Threshold of the loop of **Algorithm 3**, ι	0.001

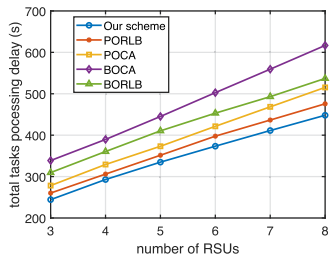


Fig. 3. Total task processing delay with different numbers of RSUs.

2) *Partial task offloading and channel allocation scheme (POCA):* the tasks of the vehicles can be partially offloaded to the RSUs that cover them, and the rest parts of the tasks are processed on the vehicles locally. Meanwhile, the channel allocation of each RSU is optimized. The edge-edge cooperation is disabled in this scheme. This scheme can embody the idea of the scheme in [13].
3) *Binary task offloading and RSU load balancing via edge-edge cooperation scheme (BORLB):* the binary task offloading and the RSU load balancing with edge-edge cooperation are both enabled in this scheme. Compared with our scheme, this scheme does not optimize the channel allocation. Instead, each RSU randomly allocates its channels to the requiring vehicles. This scheme can embody the idea of the scheme in [17].
4) *Partial task offloading and RSU load balancing via edge-edge cooperation scheme (PORLB):* the partial task offloading and the RSU load balancing with edge-edge cooperation are both enabled in this scheme. Compared with our scheme, this scheme does not optimize the channel allocation. Instead, each RSU randomly allocates its channels to the requiring vehicles.

A. Different Numbers of RSUs

As shown in Fig. 3, we increase the number of RSUs from 3 to 8, while keeping the number of high-load RSUs (50 vehicles under the coverage of each high-load RSU) as 2 unchanged. The other parameters are valued by default. It can be seen that with the increase of the number of RSUs, the total task processing delays of all the schemes are continuously rising. Among these schemes, our scheme always has the best performance under different numbers of RSUs. It can be also observed that the increase of the total delay of our scheme becomes smaller as the number of RSU increases. This is mainly because our scheme can alleviate the pressure of the high-load RSUs by offloading the tasks on them to the other RSUs. This advantage becomes more

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:56:54 UTC from IEEE Xplore. Restrictions apply.

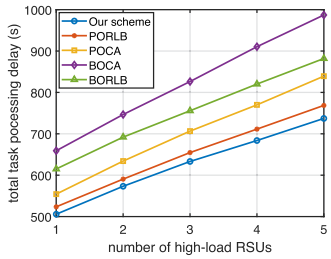


Fig. 4. Total task processing delay with different numbers of high-load RSUs.

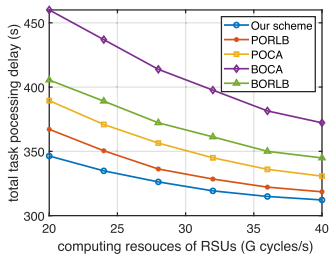


Fig. 5. Total task processing delay with different computing resources of edge servers.

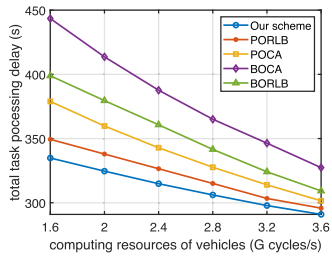


Fig. 6. Total task processing delay with different computing resources of vehicles.

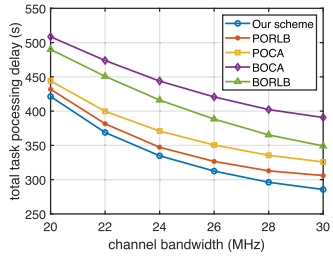


Fig. 7. Total task processing delay with different bandwidths of channels.

significant as the number of RSUs grows since the increasing RSUs enlarge the computing resources so that the load balancing can lead to a better effect. PORLB has the same trend as our scheme, but it does not optimize the channel allocation, so it performs worse especially when the number of RSUs is high because the co-channel interference becomes more severe as the number of RSUs increases. POCA and BOCA disable the edge-edge cooperation, so the load alleviation of the high-load RSUs can not be fulfilled through the edge-edge cooperation, thus resulting in system performance degradation. BOCA performs worse than POCA since the binary task offloading restricts the flexible adjustment of task scheduling between each vehicle and the RSU. BORLB performs better than BOCA but worse than PORLB. Owning to the edge-edge cooperation, it can achieve load balancing which leads to a low processing delay than BOCA. Yet, the binary offloading brings more computing delay than PORLB.

B. Different Numbers of High-Load RSUs

As depicted in Fig. 4, we increase the number of high-load RSUs (50 vehicles under the coverage of each high-load RSU) from 1 to 5, while keeping the total number of RSUs as 10 unchanged. The other parameters are set by default. It can be seen that with the increase of the number of high-load RSUs, the total task processing delays of all the schemes rise correspondingly. The total system load enlarges as the number of high-load RSUs increases. Our scheme always has the lowest total delay compared with POCA and BOCA, mainly because it leverages the edge-edge cooperation to enable load balancing among the RSUs. Thus, the pressures of the high-load RSUs can be shared with the other RSUs to improve the system performance, especially when the number of high-load RSUs is high. PORLB and BORLB are always worse than our scheme owing to the lack of channel allocation optimization and binary offloading.

C. Different Computing Resources of Edge Servers

As shown in Fig. 5, we increase the computing resources of the edge servers from 20 G cycles/s to 40 G cycles/s, while the values of the other parameters remain as default. With the increase of the computing resources of the edge servers, the total task processing delays of all the schemes decrease accordingly. Our scheme always achieves the lowest total delay owing to the joint resource management and the load balancing enabled by the edge-edge cooperation. However, compared with the other 4 schemes, it can be seen that the advantage of our scheme shrinks since the enlargement of the computing resources of the edge servers alleviates the load pressures of the RSUs, thus, the performance improvement brought by the load balancing becomes less significant. The performance of PORLB, POCA, and BORLB is worse than that of our scheme owing to the lack of channel allocation optimization, load balancing, and partial task offloading, respectively. BOCA is the worst one because neither partial of loading nor load balancing is considered.

D. Different Computing Resources of Vehicles

As described in Fig. 6, we increase the computing resources of the vehicles from 2 G cycles/s to 4.5 G cycles/s, while other parameters are set by default. It can be seen that the total task processing delays of all the schemes drop along with the increase of the computing resources of the vehicles. Our scheme is always superior to the other schemes because it jointly optimizes task offloading, channel allocation, and RSU load balancing. However, this advantage weakens as the computing resources of the vehicles rise. The main reason is that more tasks are processed on the vehicles locally as their computing resources increase, so the performance enhancement brought by the RSU load balancing declines correspondingly. Still, the performance of PORLB, POCA, and BORLB is worse than that of our scheme because of the lack of channel allocation optimization, load balancing, and partial task offloading, respectively. BORLB has a worse performance than PORLB for the reason that it doesn't leverage partial offloading.

E. Different Bandwidths of Channels

As shown in Fig. 7, we increase the bandwidth of the channels from 10 MHz to 30 MHz, while other parameters are kept as default. It can be seen that with the increase of bandwidth of the channels, the total task processing delays of all the schemes continuously reduce and our scheme always has the lowest total delay. When the channel bandwidth is small, the vehicles which offload their tasks to the RSUs will consume more time on task data transmission. Thus, the task offloading strategy will tend to schedule more tasks to be processed on the vehicles locally, thus resulting in an increase of the total delay. Our scheme is always superior to the others since it optimizes the channel allocation, and more importantly, it leverages the edge-edge cooperation to fulfill the load

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:56:54 UTC from IEEE Xplore. Restrictions apply.

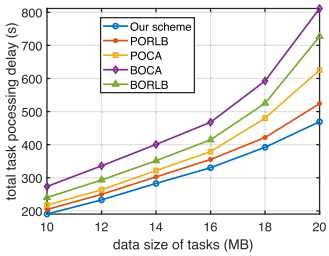


Fig. 8. Total task processing delay with different task data size of vehicles.

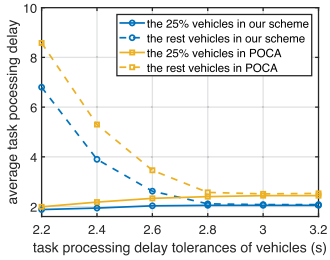


Fig. 9. Total task processing delay with different task processing delay tolerances of vehicles.

balancing, which can further promote the task offloading performance of the entire system.

F. Different Task Data Size of Vehicles

As described in Fig. 8, we increase the task data size of the vehicles from 10 MB to 20 MB, while the values of the other parameters remain as default. It can be found that the total task processing delays of all the schemes decrease with the increase of the task data size. As the task data size enlarges, the task transmission delay from the vehicles to the RSUs, the task transmission delay among the RSUs, and the task computing delay on the RSUs and the vehicle all rise correspondingly, thus increasing the load of the entire system. Our scheme can always obtain the lowest total delay, and the advantage becomes higher and higher as the system load increases. The main reason is that our scheme jointly optimizes the task offloading and channel allocation, and the load balancing via the edge-edge cooperation plays an important role in reducing the total delay. PORLB is worse than our scheme owing to the lack of channel allocation optimization, and POCA and BOCA are the worst mainly because they do not consider the edge-edge cooperation, thus, the loads of the RSU are not able to be balanced.

G. Different Task Processing Delay Tolerances of Vehicles

As depicted in Fig. 9, we increase the task processing delay tolerances of 25% vehicles in the total vehicles from 1.5 s to 4 s, and then we evaluate the average task processing delay of the 25% vehicles and the other 75% vehicles. The other parameters are valued by default. With the increase of the task processing delay tolerance, the average task processing delay of the 25% vehicles increases at first and then keep unchanged, whereas, the average delay of the rest 75% vehicles reduces at first and then keeps unchanged. This trend is the same for our scheme and POCA, because the vehicles with severer delay tolerances force the system to allocate more resources to guarantee their requirement, thus, correspondingly, leading to the performance degradation of the other vehicles because of the decrease of the rest available resources. However, our scheme is still better than POCA since it uses the edge-edge cooperation to balance the loads of the RSUs, so the performance deterioration can be restrained effectively.

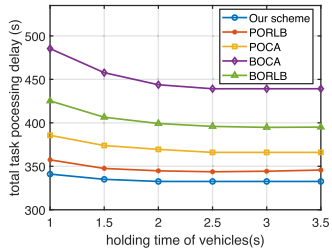


Fig. 10. Total task processing delay with different holding time of vehicles.

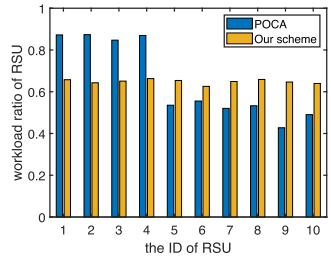


Fig. 11. RSU workloads with or without load balancing.

H. Different Holding Time of Vehicles

As shown in Fig. 10, we increase the holding time of vehicles from 1 s to 3.5 s, while keeping other parameters valued by default. We can observe that the total task processing delays of all the schemes decrease when the hold time is from 1 s to 2.2 s, and then they become stable from 2.2 s to 3.5 s. This is because when the holding time is low, the constraint $C11$ works in our optimization problem to restrict the task offloading strategy, which has to schedule the tasks to be processed on the vehicles locally, thus leading to relatively high total delay. Whereas, when the holding time becomes high, there are more tasks that can be offloaded to the RSUs, and the task offloading strategy can always meet the requirement of $C11$, so the curves become nearly flat. Our scheme still outperforms the other 4 schemes because of the joint resource management and the load balancing via the edge-edge cooperation.

I. Effect of Load Balancing

It can be seen that the load balancing via the edge-edge cooperation efficiently improves the system performance. Actually, POCA is the same as our scheme without load balancing. Thus, in Figs. 3–10, our scheme is compared with POCA by varying 8 different crucial parameters. The simulation results demonstrate that a significant performance gain can be brought by enabling load balancing via the edge-edge cooperation.

To further explain the effect of load balancing, in Fig. 11, we show the RSU workloads with load balancing (i.e. adopting our scheme) or without load balancing (i.e. adopting POCA) in a scenario consisting of 4 high load RSUs (RSU 1-4) and 6 the other RSUs (RSU 5-10). We define the workload of an RSU as a factor that measures the proportion of the computation amount of all the tasks offloaded to the RSU in the total computing resource of the RSU. It can be observed that the workloads of RSU 1-4 are much higher than those of RSU 5-10 before the load balancing, whereas, the workloads of all the RSUs are approximately equal after the load balancing.

J. Convergence Analysis of Two-Stage Iterative Algorithm

Finally, Table IV illustrates the minimum, average, and maximum number of iterations needed for the convergence of the two-stage iterative algorithm in the scenarios of Figs. 3–10. For each figure, we show the number of iterations needed for the loop of Algorithm 1, the loop of Algorithm 2, the inner loop and the outer loop of Algorithm 3.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:56:54 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
NUMBER OF ITERATIONS NEEDED FOR CONVERGENCE

Figure	Number of Iterations (Min./Avr./Max.)			
	Loop of Algorithm 1	Loop of Algorithm 2	Inner Loop of Algorithm 3	Outer Loop of Algorithm 3
Fig. 3	1/2.3/3	2/3/4	5/9.1/11	4/6.5/8
Fig. 4	1/3.3/4	2/4.1/6	5/8.7/10	4/6.3/7
Fig. 5	1/2.2/3	2/3.2/4	3/3.8/5	4/4.2/5
Fig. 6	1/2.1/3	2/3.2/4	4/5.7/8	2/3.7/5
Fig. 7	1/2.4/3	2/3/4	4/6.3/8	4/5.1/6
Fig. 8	1/2.1/3	2/2.4/5	3/5.5/7	2/4.8/6
Fig. 9	1/2.3/3	2/3.2/4	2/6.1/8	1/3.1/4
Fig. 10	1/2/3	2/3.2/4	5/5.8/7	2/3.2/4

It can be seen that the number of iterations needed by our algorithm is quite limited. The average numbers of iterations for the above four loops are maintained from 2.1 to 3.3, from 2.4 to 4.1, from 3.8 to 9.1, and from 3.1 to 6.5, respectively. We can also find that the convergence of our algorithm is impacted by the scale of the optimization problem. In Fig. 3 and Fig. 4, the rise of the number of RSUs enlarges the scale of the problem, thus further increasing the number of iterations needed by the algorithm. The results demonstrate that our algorithm only needs a small number of iterations to reach convergence.

VI. CONCLUSION

In this paper, we propose a joint partial task offloading and channel allocation scheme for a multi-RSU VEC scenario with edge-edge cooperation, which is leveraged for load balancing through RSU-RSU task offloading. The objective is to minimize the total task processing delay of all the vehicles while guaranteeing the task processing delay tolerance and the holding time of each vehicle. We use M/M/1 queues to model the task queuing and task computing processes on the RSUs, and we design an exact potential game to model the competition processing in load balancing among the RSUs. We design a two-stage iterative algorithm to decompose the optimization problem into two stages and solve the sub-problems iteratively. The superiority of our scheme is demonstrated through conducting extensive simulations in seven different scenarios and comparing our scheme with 4 other reference schemes.

APPENDIX A
PROOF OF EXACT POTENTIAL GAME

If the player mn discovers another strategy β'_{mn} can lower its utility function $u_{mn}(\beta_{mn}, \boldsymbol{\beta}_{-mn})$, it will change its strategy from β_{mn} to β'_{mn}. We call this action as a strategy improvement. Thus, we have the following definition

Defination 1: A game is recognized as an exact potential game if there exists a potential function cluster $\{\phi_{mn}\}_{n\in\mathcal{N},m\in\mathcal{M}_n}$ which matches the following condition: for each player in the game $\mathcal{G}$, any feasible unilaterally strategy improvement from $(\beta_{mn}, \boldsymbol{\beta}_{-mn})$ to $(\beta_{mn}{}', \boldsymbol{\beta}_{-mn})$ always follows the equality:

$$\begin{aligned} &u_{mn}(\beta_{nm}, \beta_{-nm}) - u_{mn}(\beta_{nm}{}', \beta_{-nm}) \\ &= \phi_{mn}(\beta_{nm}, \beta_{-nm}) - \phi_{mn}(\beta_{nm}{}', \beta_{-nm}) \end{aligned} \tag{33}$$

The above equation means that, as the player's strategy changes, the values of the potential function and the utility function change as the same.

To prove the game $\mathcal{G}$ is an exact potential game, we first construct the potential function as

$$\begin{aligned} &\phi_{mn}(\beta_{nm}, \beta_{-nm}) \\ &= \overline{\alpha}_{mn}\lambda_{mn}\left(\sum_{i\in\mathcal{N}} c_{mn}\left(t_i^{comp} + \frac{\beta_{mni}}{C_i - \xi_i + \overline{\alpha}_{mn}\lambda_{mn}c_{mn}}\right)\right. \\ &\left. + \sum_{i\in\mathcal{N}\backslash n} \beta_{mni} d_{mn}\left(\frac{1}{\theta - \xi^{LAN}} + \frac{1}{R_{ni}^{r2r}}\right)\right) \end{aligned} \tag{34}$$

To avoid verbosity, we define $t_i^{comp} = \frac{1}{C_i - \sum_{l\in\mathcal{N}}\sum_{k\in\mathcal{M}_l}(\overline{\alpha}_{kl}\beta_{kli}\lambda_{kl}c_{kl})}$, $\xi_i = \sum_{l\in\mathcal{N}}\sum_{k\in\mathcal{M}_l}(\overline{\alpha}_{kl}\beta_{kli}\lambda_{kl}c_{kl})$ and $\xi^{LAN} = \sum_{l\in\mathcal{N}}\sum_{k\in\mathcal{M}_l}\sum_{z\in\mathcal{N}\backslash l}(\overline{\alpha}_{kl}\beta_{klz}d_{kl}\lambda_{kl})$

Then, we will prove the potential function satisfies (33) when the player mn improves its strategy from β_{mn} to β'_{mn}.

The strategy improvement of the player mn can be categorized into two cases:

Case 1: it represents that the offloaded tasks of vehicle mn are further offloaded from the original RSU n that covers it to another RSU i. The strategies β_{mn} and β'_{mn} in this case can be expressed as $\beta_{mn} = \{\beta_{mnn} = 1, \beta_{mni} = 0, i \in \mathcal{N}, i \neq n\}$, $\beta'_{mn} = \{\beta_{mnn} = 0, \beta_{mni} = 1, i \in \mathcal{N}, i \neq n\}$.

Substituting (34) into (33), we have

$$\begin{aligned} &\phi(\beta_{mn}, \boldsymbol{\beta}_{-mn}) - \phi(\beta_{mn}{}', \boldsymbol{\beta}_{-mn}) \\ &= \overline{\alpha}_{mn}\lambda_{mn}\left(\sum_{i\in\mathcal{N}} c_{mn}\left(t_i^{comp} + \frac{\beta_{mni}}{C_i - \xi_i + \overline{\alpha}_{mn}\lambda_{mn}c_{mn}}\right)\right) \\ &- \overline{\alpha}_{mn}\lambda_{mn}\left(\sum_{i\in\mathcal{N}} c_{mn}\left(t_i^{comp\prime} + \frac{\beta_{mni}}{C_i - \xi'_i + \overline{\alpha}_{mn}\lambda_{mn}c_{mn}}\right)\right. \\ &\left. + \beta_{mni} d_{mn}\left(\frac{1}{\theta - \xi^{LAN\prime}} + \frac{1}{R_{ni}^{r2r}}\right)\right) \\ &= \overline{\alpha}_{mn}\lambda_{mn}\left(c_{mn}t_n^{comp} + \frac{c_{mn}}{C_n - \xi_n + \overline{\alpha}_{mn}\lambda_{mn}c_{mn}}\right. \\ &+ c_{mn}t_i^{comp} - \left(c_{mn}t_n^{comp\prime} + c_{mn}t_i^{comp\prime}\right. \\ &\left. + \frac{c_{mn}}{C_i - \xi'_i + \overline{\alpha}_{mn}\lambda_{mn}c_{mn}}\right) \\ &- \beta_{mni} d_{mn}\left(\frac{1}{\theta - \xi^{LAN\prime}} + \frac{1}{R_{ni}^{r2r}}\right)\Big) \\ &= \overline{\alpha}_{mn}\lambda_{mn}\left(c_{mn}\left(t_n^{comp} - t_i^{comp\prime}\right)\right. \\ &\left. - \beta_{mni} d_{mn}\left(\frac{1}{\theta - \xi^{LAN\prime}} + \frac{1}{R_{ni}^{r2r}}\right)\right) \\ &= u(\beta_{mn}, \boldsymbol{\beta}_{-mn}) - u(\beta_{mn}{}', \boldsymbol{\beta}_{-mn}) \end{aligned} \tag{35}$$

Case 2: It represents that the offloaded tasks of vehicle mn are further offloaded from RSU i to another RSU j, where $i, j \neq n$. The strategies β_{mn} and β'_{mn} in this case can be expressed as $\beta_{mn} = \{\beta_{mni} = 1, \beta_{mnj} = 0, i, j \in \mathcal{N}\backslash n, j \neq i\}$, $\beta'_{mn} = \{\beta_{mni} = 0, \beta_{mnj} = 1, i, j \in \mathcal{N}\backslash n, j \neq i\}$.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:56:54 UTC from IEEE Xplore. Restrictions apply.

Substituting (34) into (33), we have

$$\begin{aligned}
&\phi(\beta_{mn},\beta_{-mn})-\phi(\beta'_{mn},\beta_{-mn})\\
&=\overline{\alpha}_{mn}\lambda_{mn}\Bigg(\sum_{i\in\mathcal{N}}c_{mn}\left(t_i^{comp}+\frac{1}{C_i-\xi_i+\overline{\alpha}_{mn}\lambda_{mn}c_{mn}}\right)\\
&-\sum_{i\in\mathcal{N}}c_{mn}\left(t_i^{comp\prime}+\frac{\beta_{mni}}{C_i-\xi'_i+\overline{\alpha}_{mn}\lambda_{mn}c_{mn}}\right)\\
&+\beta_{mni}d_{mn}\left(\frac{1}{\theta-\xi^{LAN}}+\frac{1}{R_{ni}^{r2r}}\right)\\
&-\beta_{mnj}d_{mn}\left(\frac{1}{\theta-\xi^{LAN\prime}}+\frac{1}{R_{nj}^{r2r}}\right)\Bigg)\\
&=\overline{\alpha}_{mn}\lambda_{mn}\Bigg(c_{mn}\Bigg(\left(t_i^{comp}+\frac{1}{C_i-\xi_i+\overline{\alpha}_{mn}\lambda_{mn}c_{mn}}\right.\\
&\left.+t_j^{comp}\right)-\left(t_i^{comp\prime}+t_j^{comp\prime}+\frac{1}{C_j-\xi'_j+\overline{\alpha}_{mn}\lambda_{mn}c_{mn}}\right)\Bigg)\\
&+d_{mn}\left(\frac{\beta_{mni}}{R_{ni}^{r2r}}-\frac{\beta_{mnj}}{R_{nj}^{r2r}}\right)\Bigg)\\
&=\overline{\alpha}_{mn}\lambda_{mn}\left(c_{mn}(t_i^{comp}-t_j^{comp\prime})+d_{mn}\left(\frac{\beta_{mni}}{R_{ni}^{r2r}}-\frac{\beta_{mnj}}{R_{nj}^{r2r}}\right)\right)\\
&=u(\beta_{mn},\beta_{-mn})-u(\beta'_{mn},\beta_{-mn}) \qquad (36)
\end{aligned}$$

Thus, it can be found from the above (35) and (36) that the equality (33) always hold in the two cases. Therefore, the game $\mathcal{G}$ is an exact potential game.

Appendix B
Proof of Existence of Nash Equilibrium

We have the following definition:

Defination 2: A strategy set $\boldsymbol{\beta}^*=\{\beta^*_{mn},\boldsymbol{\beta}^*_{-mn}\}$ is a NE for the exact game $\mathcal{G}$ if it satisfies the following relationship for every player in the game:

$$u(\beta^*_{mn},\boldsymbol{\beta}^*_{-mn})\le u(\beta_{mn},\boldsymbol{\beta}^*_{-mn}) \qquad (37)$$

The above inequality implies that a game reaches the NE when each player in the game no longer can decrease its utility function by unilaterally changing its own strategy.

We have already proven the game $\mathcal{G}$ is an exact potential game. Moreover, the strategy set $\boldsymbol{\beta}$ is a finite set. As proved in [11], every exact potential game with finite strategy sets always has a NE and provides the finite improvement property. Thus, the existence of the NE of the game $\mathcal{G}$ is proved.

References

[1] L. Liu, C. Chen, Q. Pei, S. Maharjan, and Y. Zhang, "Vehicular edge computing and networking: A survey," *Mobile Netw. Appl.*, vol. 26, no. 3, pp. 1145–1168, 2021.

[2] C.-C. Lin, D.-J. Deng, and C.-C. Yao, "Resource allocation in vehicular cloud computing systems with heterogeneous vehicles and roadside units," *IEEE Internet Things J.*, vol. 5, no. 5, pp. 3692–3700, Oct. 2018.

[3] M. Najm, M. Patra, and V. Tamarapalli, "Cost-and-delay aware dynamic resource allocation in federated vehicular clouds," *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 6159–6171, Jun. 2021.

[4] R. I. Meneguette and A. Boukerche, "Vehicular clouds leveraging mobile urban computing through resource discovery," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 6, pp. 2640–2647, Jun. 2020.

[5] B. Yang, D. Wu, H. Wang, Y. Gao, and R. Wang, "Two-layer stackelberg game-based offloading strategy for mobile edge computing enhanced FiWi access networks," *IEEE Trans. Green Commun. Netw.*, vol. 5, no. 1, pp. 457–470, Mar. 2021.

[6] W. Fan, Y. Liu, B. Tang, F. Wu, and Z. Wang, "Computation offloading based on cooperations of mobile edge computing-enabled base stations," *IEEE Access*, vol. 6, pp. 22622–22633, 2018.

[7] W. Fan, L. Yao, J. Han, F. Wu, and Y. Liu, "Game-based multitype task offloading among mobile-edge-computing-enabled base stations," *IEEE Internet Things J.*, vol. 8, no. 24, pp. 17691–17704, Dec. 2021.

[8] W. Fan et al., "DNN deployment, task offloading, and resource allocation for joint task inference in IIoT," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1634–1646, 2023.

[9] W. Fan et al., "Collaborative service placement, task scheduling, and resource allocation for task offloading with edge-cloud cooperation," *IEEE Trans. Mobile Comput.*, early access, doi: 10.1109/TMC.2022.3219261.

[10] D. Gross, *Fundamentals of Queueing Theory*. Hoboken, NJ, USA: Wiley, 2008.

[11] D. Monderer and L. S. Shapley, "Potential games," *Games Econ. Behav.*, vol. 14, pp. 124–143, 1996. .

[12] Y.-J. Ku, P.-H. Chiang, and S. Dey, "Real-time QoS optimization for vehicular edge computing with off-grid roadside units," *IEEE Trans. Veh. Technol.*, vol. 69, no. 10, pp. 11975–11991, Oct. 2020.

[13] B. Shang, L. Liu, and Z. Tian, "Deep learning-assisted energy-efficient task offloading in vehicular edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 70, no. 9, pp. 9619–9624, Sep. 2021.

[14] Z. Zhou, J. Feng, Z. Chang, and X. Shen, "Energy-efficient edge computing service provisioning for vehicular networks: A consensus ADMM approach," *IEEE Trans. Veh. Technol.*, vol. 68, no. 5, pp. 5087–5099, May 2019.

[15] Z. Ning et al., "Intelligent edge computing in internet of vehicles: A joint computation offloading and caching solution," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2212–2225, Apr. 2021.

[16] S. Li, N. Zhang, H. Chen, S. Lin, O. A. Dobre, and H. Wang, "Joint road side units selection and resource allocation in vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 70, no. 12, pp. 13190–13204, Dec. 2021.

[17] J. Zhang, H. Guo, J. Liu, and Y. Zhang, "Task offloading in vehicular edge computing networks: A load-balancing solution," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 2092–2104, Feb. 2020.

[18] W. Fan, J. Liu, M. Hua, F. Wu, and Y. Liu, "Joint task offloading and resource allocation for multi-access edge computing assisted by parked and moving vehicles," *IEEE Trans. Veh. Technol.*, vol. 71, no. 5, pp. 5314–5330, May 2022.

[19] X. Zhu, Y. Luo, A. Liu, M. Z. A. Bhuiyan, and S. Zhang, "Multiagent deep reinforcement learning for vehicular computation offloading in IoT," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9763–9773, Jun. 2021.

[20] W. Fan et al., "Joint task offloading and resource allocation for vehicular edge computing based on V2I and V2V modes," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 4277–4292, Apr. 2023.

[21] X. Kong et al., "Deep reinforcement learning based energy efficient edge computing for internet of vehicles," *IEEE Trans. Ind. Informat.*, vol. 18, no. 9, pp. 6308–6316, Sep. 2022.

[22] X. Huang, L. He, and W. Zhang, "Vehicle speed aware computing task offloading and resource allocation based on multi-agent reinforcement learning in a vehicular edge computing network," in *Proc. IEEE Int. Conf. Edge Comput.*, 2020, pp. 1–8.

[23] X. Li, Y. Dang, M. Aazam, X. Peng, T. Chen, and C. Chen, "Energy-efficient computation offloading in vehicular edge cloud computing," *IEEE Access*, vol. 8, pp. 37632–37644, 2020.

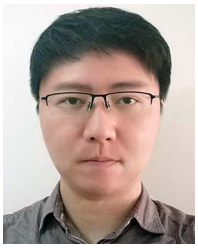
[24] Z. Wang, S. Zheng, Q. Ge, and K. Li, "Online offloading scheduling and resource allocation algorithms for vehicular edge computing system," *IEEE Access*, vol. 8, pp. 52428–52442, 2020.

[25] M. Liwang, J. Wang, Z. Gao, X. Du, and M. Guizani, "Game theory based opportunistic computation offloading in cloud-enabled IoV," *IEEE Access*, vol. 7, pp. 32551–32561, 2019.

[26] X. Hou et al., "Reliable computation offloading for edge-computing-enabled software-defined IoV," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7097–7111, Aug. 2020.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:56:54 UTC from IEEE Xplore. Restrictions apply.

[27] Q. Shen, B.-J. Hu, and E. Xia, "Dependency-Aware task offloading and service caching in vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13182–13197, Dec. 2022.
[28] H. Gao, W. Huang, T. Liu, Y. Yin, and Y. Li, "PPO2: Location privacy-oriented task offloading to edge computing using reinforcement learning for intelligent autonomous transport systems," *IEEE Trans. Intell. Transp. Syst.*, early access, doi: 10.1109/TITS.2022.3169421.
[29] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, Oct.–Dec. 2017.
[30] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint load balancing and offloading in vehicular edge computing and networks," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4377–4387, Jun. 2019.
[31] D. Bertsekas and R. Gallager, *Data networks*. Nashua, NH, USA: Athena Sci., 2021.
[32] Q. Yuan, J. Li, H. Zhou, T. Lin, G. Luo, and X. Shen, "A joint service migration and mobility optimization approach for vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 8, pp. 9041–9052, Aug. 2020.
[33] R. Q. Hu et al., "Twin-timescale artificial intelligence aided mobility-aware edge caching and computing in vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 4, pp. 3086–3099, Apr. 2019.
[34] T. X. Tran and D. Pompili, "Joint task offloading and resource allocation for multi-server mobile-edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 1, pp. 856–868, Jan. 2019.
[35] L. Liang, G. Y. Li, and W. Xu, "Resource allocation for D2D-enabled vehicular communications," *IEEE Trans. Commun.*, vol. 65, no. 7, pp. 3186–3197, Jul. 2017.
[36] S. Long, W. Long, Z. Li, K. Li, Y. Xia, and Z. Tang, "A game-based approach for cost-aware task assignment with QoS constraint in collaborative edge and cloud environments," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1629–1640, Jul. 2021.
[37] S. Boyd, S. P. Boyd, and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.
[38] C. Wang, C. Liang, F. R. Yu, Q. Chen, and L. Tang, "Computation offloading and resource allocation in wireless cellular networks with mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 16, no. 8, pp. 4924–4938, Aug. 2017.
[39] J. Zhang et al., "Energy-latency tradeoff for energy-aware offloading in mobile edge computing networks," *IEEE Internet Things J.*, vol. 5, no. 4, pp. 2633–2645, Aug. 2018.



Wenhao Fan (Member, IEEE) received the B.E. and Ph.D. degrees from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2008 and 2013, respectively. He is currently an Associate Professor with the School of Electronic Engineering, BUPT. He has authored/coauthored more than 60 academic papers and has held more than 30 patents for invention. His main research interests include edge computing, edge intelligence, computer networks, AI-empowered IoV, and IoT.



Mingyu Hua received the B.E. degree from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2020. He is currently a Postgraduate Student with the School of Electronic Engineering, BUPT. His main research focuses on edge computing in IoV applications.



Yaoyin Zhang received the B.E. degree from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2021. She is currently a Postgraduate Student with the School of Electronic Engineering, BUPT. His main research focuses on edge computing in IoV applications.



Yi Su received the B.E. degree from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2018. He is currently working toward the Ph.D. degree with the School of Electronic Engineering, BUPT. His main research interests include edge/cloud computing, edge intelligence, game theory, and IoV technologies.



Xuewei Li received the B.E. degree from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2022. He is currently a Postgraduate Student with the School of Electronic Engineering, BUPT. His main research focuses on edge computing in IoV applications.



Fan Wu (Member, IEEE) received the B.S. degree in telecommunication engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 2004, and the Ph.D. degree from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2009. She is currently an Associate Professor with the School of Electronic Engineering, BUPT. Her research interests include IoT search and wireless sensor networks.



Yuan'an Liu (Member, IEEE) received the B.E., M.Eng., and Ph.D. degrees in electrical engineering from the University of Electronic Science and Technology, Chengdu, China, in 1984, 1989, and 1992, respectively. He is currently a Professor with the School of Electronic Engineering, Beijing University of Posts and Telecommunications (BUPT), where he is also Dean with the Beijing Key Laboratory of Work Safety Intelligent Monitoring. He is a Fellow of the Institution of Engineering and Technology, U.K., Vice Chairman of Electromagnetic Environment and Safety of the China Communication Standards Association, Vice Director of the Wireless and Mobile Communication Committee, Communication Institute of China, Senior Member of the Electronic Institute of China. His main research interests include wireless communications, electromagnetic compatibility, and pervasive computing.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:56:54 UTC from IEEE Xplore. Restrictions apply.